

Sonderheft II

Tech Club Agentic OS

Blueprint für ein EU-AI-Act-ready agentisches Betriebssystem

Seite 1

Warum ein neues Betriebssystem für Agenten?

Künstliche Intelligenz entwickelt sich von einfachen Chatbots zu komplexen agentischen Systemen.

Ein klassischer Chatbot beantwortet Fragen.

Ein Agent kann zusätzlich:

- Ziele verfolgen,
- Werkzeuge nutzen,
- Informationen abrufen,
- Entscheidungen vorbereiten,
- Aufgaben ausführen,
- mit anderen Agenten kommunizieren,
- und aus Ergebnissen lernen.

Dadurch entsteht eine neue technische Herausforderung.

Wir benötigen nicht nur bessere Modelle.

Wir benötigen ein **Betriebssystem für Agenten**.

Was ist ein Agentic OS?

Ein Agentic OS ist eine Laufzeitumgebung,

in der KI-Agenten sicher,

kontrolliert,

nachvollziehbar

und koordiniert arbeiten können.

Es verbindet:

- Agenten,
- Skills,
- Speicher,
- Datenquellen,
- APIs,
- MCP-Server,
- Benutzer,
- Berechtigungen,
- Risikoanalyse,
- Protokollierung
- und menschliche Aufsicht

zu einem gemeinsamen System.

Warum reicht ein normaler Chat nicht aus?

Ein Chat-System ist meist dialogorientiert.

Ein Agentensystem ist handlungsorientiert.

Das bedeutet:

Ein Agent kann nicht nur antworten.

Er kann Prozesse starten.

Zum Beispiel:

- eine Datei analysieren,
- Daten aus einer API laden,
- einen Workflow vorbereiten,
- eine Aufgabe an einen anderen Agenten übergeben,
- ein Dokument erzeugen,
- ein Ticket erstellen,
- oder eine Entscheidung vorschlagen.

Sobald KI-Systeme handeln können,

entstehen neue Anforderungen an Sicherheit, Kontrolle und Dokumentation.

Die Grundfrage

Die wichtigste Frage lautet nicht:

Was kann ein Agent tun?

Sondern:

Was darf ein Agent tun, unter welchen Bedingungen, mit welchen Daten, für welchen Zweck und mit welcher menschlichen Kontrolle?

Diese Frage bildet den Kern des Tech Club Agentic OS.

EU-AI-Act-ready von Anfang an

Der EU AI Act arbeitet risikobasiert.

Das bedeutet:

Nicht jede KI-Anwendung wird gleich behandelt.

Entscheidend sind:

- Zweck,
- Einsatzbereich,

- betroffene Nutzergruppe,
- mögliche Auswirkungen,
- Grundrechte,
- Transparenz,
- Aufsicht,
- Dokumentation
- und Risiko.

Deshalb wird Tech Club Agentic OS nicht erst später um Compliance erweitert.

Compliance ist Teil der Grundarchitektur.

Zwei Kernel arbeiten parallel

Tech Club Agentic OS besteht aus zwei zentralen Kernen.

1. Agent Runtime Kernel

Er steuert:

- Wahrnehmung,
- Kontext,
- Ziele,
- Skills,
- Gedächtnis,
- Kommunikation,
- Aufgaben
- und Ausführung.

2. Compliance Kernel

Er prüft:

- Risikoklasse,
- Berechtigungen,
- verbotene Praktiken,
- Datenqualität,
- Transparenz,
- menschliche Aufsicht,
- Logs,

- Modellversionen,
- Vorfälle
- und Dokumentationspflichten.

Beide Kernel laufen parallel.

Kein Agent handelt außerhalb dieser Architektur.

Grundprinzip

User Request



Intent Understanding



Risk Classification



Policy Check



Context Retrieval



Skill Selection



Human Oversight, falls notwendig



Agent Action



Audit Log



Result Review



Memory Update

Jede Aktion wird geprüft,

ausgeführt,

dokumentiert

und bei Bedarf eskaliert.

Ziel des Sonderhefts

Dieses Sonderheft definiert die technische Grundlage des Tech Club Agentic OS.

Es beschreibt:

- die Syntax,
 - den Compliance-Kernel,
 - die wichtigsten Agentenalgorithmien,
 - die Sicherheitsarchitektur,
 - die Multiagentenkoordination,
 - die Verbindung zu MCP-Servern,
 - und den Aufbau einer EU-tauglichen Runtime.
-

Tech Club Erkenntnis

Ein agentisches System darf nicht nur leistungsfähig sein.

Es muss auch:

- erklärbar,
- kontrollierbar,
- auditierbar,
- sicher,
- transparent
- und menschlich beaufsichtigt sein.

Tech Club Agentic OS verfolgt deshalb ein klares Ziel:

Kein Agent ohne Risikoklasse.

Kein Skill ohne Berechtigung.

Keine kritische Entscheidung ohne menschliche Aufsicht.

Keine Aktion ohne Log.

Kein Modell ohne Dokumentation.

So entsteht kein unkontrolliertes Experimentalsystem,

sondern eine regulierbare, überprüfbare und europataugliche Agentenarchitektur.

Sonderheft II

Tech Club Agentic OS

Blueprint einer EU-AI-Act-ready Agent Runtime

Seite 1

Agentic OS – Das Betriebssystem der nächsten Generation

Von Programmen zu intelligenten Agenten

Die Geschichte der Computer begann mit Programmen.

Später entstanden Betriebssysteme.

Dann folgten das Internet,

Cloud Computing,

Smartphones

und schließlich Künstliche Intelligenz.

Heute beginnt eine neue Entwicklungsstufe:

Agentische Systeme.

Ein Agent beantwortet nicht nur Fragen.

Er kann beobachten,

planen,

Werkzeuge verwenden,

mit anderen Agenten kommunizieren,

Aufgaben koordinieren

und Ziele Schritt für Schritt verfolgen.

Damit verändert sich die Rolle des Betriebssystems grundlegend.

Warum ein Agentic Operating System?

Ein klassisches Betriebssystem verwaltet:

- Prozesse
- Speicher
- Dateien
- Geräte
- Netzwerk
- Benutzer

Ein Agentic Operating System verwaltet zusätzlich:

- Agenten
- Ziele
- Fähigkeiten (Skills)
- Wissen
- Kontext
- Entscheidungen
- Risiken
- Kommunikation
- Compliance
- menschliche Aufsicht

Dadurch entsteht eine neue Systemschicht zwischen Mensch, KI und digitaler Infrastruktur.

Die Philosophie von Tech Club

Tech Club versteht einen Agenten nicht als autonomes Wesen.

Ein Agent ist ein **digitaler Mitarbeiter**.

Er besitzt:

- eine Aufgabe,
- klar definierte Fähigkeiten,
- begrenzte Berechtigungen,
- nachvollziehbare Entscheidungen
- und überprüfbare Verantwortung.

Jeder Agent arbeitet innerhalb eines kontrollierten Systems.

Nicht außerhalb davon.

Die Grundarchitektur

Das Agentic OS besteht aus zwei parallel arbeitenden Kernen.

Human

↓

Agent Runtime Kernel

↔

Compliance Kernel

↓

Skills

↓

Memory

↓

MCP Server

↓

APIs

↓

Data Graph



Tools



Audit

Während der Runtime Kernel Aufgaben ausführt,
überwacht der Compliance Kernel jede einzelne Aktion.

Grundprinzip

Jede Agentenaktion folgt demselben Lebenszyklus.

Beobachten



Verstehen



Kontext analysieren



Risiko bewerten



Skill auswählen



Aktion planen

↓

Menschliche Freigabe (falls erforderlich)

↓

Ausführen

↓

Dokumentieren

↓

Lernen

Dadurch wird jede Entscheidung nachvollziehbar.

AI by Design – Compliance by Design

Tech Club integriert Sicherheit nicht nachträglich.

Sie ist Bestandteil der Architektur.

Deshalb besitzt jeder Agent:

- eine Identität,
- eine Risikoklasse,
- erlaubte Fähigkeiten,
- dokumentierte Modelle,
- nachvollziehbare Logs,
- definierte Verantwortlichkeiten,
- und menschliche Kontrollmechanismen.

Dadurch kann das gesamte System regulatorische Anforderungen von Anfang an berücksichtigen.

Zielsetzung

Das Agentic OS soll nicht möglichst autonome Agenten erzeugen.

Es soll **vertrauenswürdige Agenten** erzeugen.

Agenten,

die:

- ✓ transparent arbeiten
 - ✓ nachvollziehbar entscheiden
 - ✓ sicher kommunizieren
 - ✓ Wissen verantwortungsvoll nutzen
 - ✓ Menschen unterstützen
 - ✓ und sich in bestehende Unternehmens- und Behördenprozesse integrieren lassen.
-

Vision

In Zukunft werden Millionen spezialisierter Agenten gleichzeitig arbeiten.

Nicht als Ersatz des Menschen.

Sondern als intelligente Werkzeuge.

Das Agentic OS wird dabei dieselbe Rolle übernehmen,

die klassische Betriebssysteme seit Jahrzehnten für Computer übernehmen:

Es organisiert,

koordiniert,

schützt,

überwacht

und verbindet alle Komponenten zu einem gemeinsamen, sicheren System.

Tech Club Erkenntnis

Die nächste Generation der Informatik wird nicht mehr nur Programme verwalten.

Sie wird **Agenten verwalten**.

Deshalb benötigt Europa eine Architektur,

die technische Leistungsfähigkeit mit Transparenz,

Sicherheit,

Datenschutz,

Nachvollziehbarkeit

und menschlicher Verantwortung verbindet.

Tech Club Agentic OS wurde genau für diese Aufgabe konzipiert.

Es ist kein Chatbot.

Es ist keine einzelne KI.

Es ist die Blueprint-Architektur für ein sicheres, auditierbares und EU-konformes agentisches Betriebssystem der nächsten Generation.

Sonderheft II

Tech Club Agentic OS

Seite 2

Der Compliance Kernel – Das Gewissen des Agentic OS

Sicherheit ist kein Modul.

Sie ist Bestandteil jeder Entscheidung.

In klassischen Softwaresystemen wird Sicherheit häufig als zusätzliche Funktion betrachtet.

Erst wird entwickelt.

Später werden Berechtigungen,

Protokollierung

und Datenschutz ergänzt.

Für agentische Systeme reicht dieses Vorgehen nicht aus.

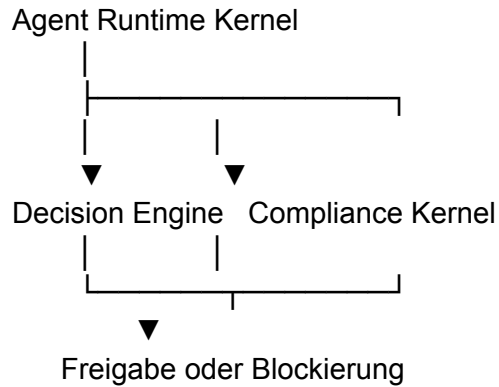
Ein Agent kann selbstständig handeln.

Deshalb muss jede Handlung bereits vor ihrer Ausführung überprüft werden.

Tech Club integriert Compliance deshalb direkt in den Systemkern.

Zwei Kernel arbeiten gleichzeitig

Das Agentic OS besitzt zwei vollständig getrennte, aber parallel arbeitende Kerne.



Der Runtime Kernel plant Aktionen.

Der Compliance Kernel bewertet,

ob diese Aktion überhaupt ausgeführt werden darf.

Aufgaben des Compliance Kernels

Der Kernel überprüft bei jeder Agentenaktion gleichzeitig:

Identität

Wer führt die Aktion aus?

Berechtigung

Darf dieser Agent diese Aktion ausführen?

Zweckbindung

Entspricht die Nutzung dem vorgesehenen Zweck?

Risikoklasse

Welcher AI-Act-Kategorie gehört dieser Workflow an?

Datenschutz

Werden personenbezogene Daten verarbeitet?

Sind die notwendigen Berechtigungen vorhanden?

Transparenz

Muss der Nutzer darüber informiert werden,
dass KI eingesetzt wird?

Menschliche Aufsicht

Ist eine Freigabe erforderlich,
bevor die Aktion ausgeführt werden darf?

Compliance wird zum Datenstrom

Jede Anfrage durchläuft denselben Prüfpfad.

User Request

↓

Intent Analysis

↓

Risk Classification



Policy Evaluation



Permission Check



GDPR Check



Transparency Check



Human Oversight



Decision



Execution



Audit Log

Keine Aktion umgeht diese Pipeline.

Policy Engine

Der Compliance Kernel arbeitet regelbasiert.

Beispiele:

IF

Risk == High

THEN

Human Approval Required

IF

Personal Data == TRUE

AND

Consent == FALSE

THEN

Block Request

IF

Skill not Authorized

THEN

Reject Execution

Dadurch bleiben Entscheidungen reproduzierbar.

Dynamische Richtlinien

Nicht jede Organisation besitzt dieselben Regeln.

Deshalb verwaltet Tech Club Richtlinien getrennt vom Agenten.

Zum Beispiel:

- Unternehmensrichtlinien
- Behördenrichtlinien
- Hochschulrichtlinien
- Datenschutzregeln

- Sicherheitsregeln
- Projektrichtlinien

Neue Regeln können ergänzt werden,
ohne den Agenten selbst zu verändern.

Compliance als kontinuierlicher Prozess

Compliance endet nicht nach einer erfolgreichen Prüfung.

Auch während der Laufzeit beobachtet der Kernel:

- ungewöhnliche Aktivitäten,
- Regelverletzungen,
- neue Risiken,
- fehlerhafte Agenten,
- inkonsistente Daten,
- unerwartete Ergebnisse.

Dadurch entsteht ein kontinuierliches Monitoring.

Vorteile der Architektur

Der Compliance Kernel ermöglicht:

- ✓ AI-Act-konforme Entscheidungen
- ✓ DSGVO-Unterstützung
- ✓ nachvollziehbare Prüfpfade
- ✓ reproduzierbare Entscheidungen
- ✓ zentrale Richtlinienverwaltung
- ✓ sichere Multiagentensysteme
- ✓ Auditierbarkeit

✓ organisatorische Kontrolle

Tech Club Erkenntnis

Ein leistungsfähiger Agent allein genügt nicht.

Ein vertrauenswürdiger Agent benötigt eine Instanz,

die jede Handlung überprüft,

Risiken bewertet,

Richtlinien anwendet,

menschliche Kontrolle ermöglicht

und jede Entscheidung nachvollziehbar dokumentiert.

Der Compliance Kernel ist deshalb nicht nur eine Sicherheitsfunktion.

Er ist das institutionelle Gedächtnis und das regulatorische Gewissen des gesamten Tech Club Agentic OS.

Er verbindet Technik, Recht, Datenschutz und Governance zu einer gemeinsamen, kontinuierlich arbeitenden Kontrollschicht – ohne den eigentlichen Agenten in seiner Arbeitsweise unnötig einzuschränken.

Sonderheft II

Tech Club Agentic OS

Seite 3

Agent Syntax – Die gemeinsame Sprache aller Agenten

Warum Agenten eine eigene Systemsprache benötigen

Programme kommunizieren über Programmiersprachen.

Betriebssysteme kommunizieren über Systemaufrufe.

Netzwerke kommunizieren über Protokolle.

Agenten benötigen ebenfalls eine gemeinsame Sprache.

Nicht nur für den Menschen.

Sondern auch untereinander.

Diese Sprache nennt Tech Club:

Agent Syntax (ASX)

Sie beschreibt nicht nur Befehle,

sondern Absichten,

Ziele,

Rollen,

Kontext,

Berechtigungen,

Risiken,

Werkzeuge

und Ergebnisse.

Vom Prompt zur Agent Instruction

Ein normaler Prompt lautet beispielsweise:

"Erstelle eine Zusammenfassung dieses Dokuments."

Ein Agent benötigt wesentlich mehr Informationen.

Nicht nur:

Was?

Sondern gleichzeitig:

- Warum?
- Für wen?
- Mit welchen Daten?
- Mit welchen Werkzeugen?
- Unter welchen Regeln?
- Mit welchem Risiko?
- Wie wird das Ergebnis bewertet?

Dadurch entsteht eine strukturierte Agent Instruction.

Die Grundstruktur

Jede Agentenanweisung besitzt dieselbe Architektur.

INTENT

↓

CONTEXT

↓

GOAL



CONSTRAINTS



RESOURCES



SKILLS



RISK



EXECUTION PLAN



OUTPUT



MEMORY UPDATE

Dadurch verstehen alle Agenten dieselbe Sprache.

Beispiel

Nicht:

Analysiere dieses Dokument.

Sondern:

INTENT:

Analyse

GOAL:

Technische Zusammenfassung erstellen

CONTEXT:

Projekt Alpha

INPUT:

Dokument Version 4

SKILLS:

Summarization

Knowledge Graph

Citation Engine

CONSTRAINTS:

Keine personenbezogenen Daten speichern

RISK:

Low

OUTPUT:

Markdown Report

STORE:

Project Memory

Diese Struktur ist sowohl für Menschen als auch für Agenten lesbar.

Syntax ist unabhängig vom Modell

Die Agent Syntax gehört **nicht** zu einem bestimmten LLM.

Sie kann verwendet werden mit:

- GPT

- Claude
- Gemini
- Mistral
- Llama
- Qwen
- lokalen Modellen
- spezialisierten Agenten

Dadurch bleibt das Agentic OS modellunabhängig.

Jede Aktion besitzt dieselben Bausteine

Unabhängig von der Aufgabe.

Zum Beispiel:

Recherche

↓

Analyse

↓

Programmierung

↓

Übersetzung

↓

Planung

↓

Simulation

↓

API-Aufruf

↓

Dokumentation

Alle folgen derselben Syntax.

Maschinenlesbar und menschenlesbar

Die Syntax besitzt zwei Darstellungen.

Human Layer

Für Entwickler und Benutzer.

Leicht verständlich.

Gut dokumentierbar.

Runtime Layer

Für Agenten.

Strukturierte Objekte.

JSON

YAML

oder interne Runtime-Objekte.

Dadurch bleibt dieselbe Bedeutung erhalten,

unabhängig von der technischen Darstellung.

Vorteile einer gemeinsamen Syntax

Eine standardisierte Agentensprache ermöglicht:

- ✓ Wiederverwendbare Workflows
 - ✓ Austauschbare Agenten
 - ✓ Modellunabhängigkeit
 - ✓ Sichere Skill-Auswahl
 - ✓ Nachvollziehbare Entscheidungen
 - ✓ Automatische Dokumentation
 - ✓ Einheitliche Logs
 - ✓ Einfache Erweiterbarkeit
-

Vorbereitung auf Multiagentensysteme

Wenn zehn Agenten zusammenarbeiten,

müssen sie dieselbe Sprache sprechen.

Nicht dieselbe Programmiersprache.

Sondern dieselbe **Operationssprache**.

Agent A muss erkennen können:

- Ziel
- Priorität
- Kontext
- Risiko
- benötigte Skills
- erwartetes Ergebnis

ohne den internen Code von Agent B zu kennen.

Genau diese Interoperabilität schafft die Agent Syntax.

Tech Club Erkenntnis

Die wichtigste Schnittstelle zukünftiger KI-Systeme wird nicht die API sein.

Sondern die Sprache,

mit der Agenten denken,

planen,

kommunizieren

und handeln.

Tech Club Agent Syntax bildet deshalb die universelle Betriebssprache des gesamten Agentic OS.

Sie verbindet Menschen, KI-Modelle, Skills, MCP-Server, APIs und Multiagentensysteme zu einer gemeinsamen, standardisierten und auditierbaren Kommunikationsschicht.

Nicht Prompts stehen im Mittelpunkt.

Sondern strukturierte Absichten, nachvollziehbare Entscheidungen und sichere Zusammenarbeit.

Sonderheft II

Tech Club Agentic OS

Seite 4

Algorithmus I – Observe · Interpret · Decide · Act

Der universelle Lebenszyklus jedes Agenten

Jedes intelligente Lebewesen folgt einem einfachen Grundprinzip.

Es beobachtet.

Es interpretiert.

Es entscheidet.

Es handelt.

Anschließend beobachtet es erneut.

Tech Club Agentic OS übernimmt genau diesen natürlichen Zyklus als grundlegenden Runtime-Algorithmus.

Nicht als einzelne Funktion.

Sondern als Herzstück jedes Agenten.

Der OIDA-Zyklus

Jeder Agent arbeitet kontinuierlich in einem geschlossenen Kreislauf.

Observe

↓

Interpret

↓

Decide

↓

Act

↓

Evaluate

↓

Learn

↓

Observe

Dieser Zyklus endet niemals,
solange der Agent aktiv ist.

Phase 1 – Observe

Der Agent sammelt Informationen.

Mögliche Quellen:

- Benutzer
- Sensoren
- APIs
- MCP Server
- Wissensgraph

- Dateien
- Datenbanken
- andere Agenten
- Systemstatus

In dieser Phase werden keine Entscheidungen getroffen.

Es wird ausschließlich beobachtet.

Phase 2 – Interpret

Nun beginnt das eigentliche Verständnis.

Der Agent analysiert:

- Kontext
- Ziel
- Priorität
- Beziehungen
- vorhandenes Wissen
- Unsicherheit
- Vertrauensniveau

Dabei entsteht ein internes Situationsmodell.

Nicht jede Information ist gleich wichtig.

Phase 3 – Decide

Erst jetzt beginnt die Entscheidungsphase.

Der Agent beantwortet Fragen wie:

Welche Aufgabe besitzt die höchste Priorität?

Welche Skills werden benötigt?

Welche Risiken bestehen?

Ist menschliche Freigabe erforderlich?

Welche Daten dürfen verwendet werden?

Welche Strategie besitzt die höchste Erfolgswahrscheinlichkeit?

Alle Entscheidungen bleiben nachvollziehbar.

Phase 4 – Act

Erst nach erfolgreicher Prüfung erfolgt die Ausführung.

Zum Beispiel:

- API aufrufen
- Dokument erzeugen
- Agent kontaktieren
- Daten analysieren
- Datei speichern
- Workflow starten
- Bericht erstellen

Jede Aktion erhält eine eindeutige Runtime-ID.

Phase 5 – Evaluate

Nach jeder Aktion bewertet sich der Agent selbst.

Fragen:

War das Ziel erreicht?

Sind Fehler aufgetreten?

Hat sich der Kontext verändert?

War die Entscheidung sinnvoll?

Sind neue Risiken entstanden?

Dadurch entsteht kontinuierliches Feedback.

Phase 6 – Learn

Nicht jedes Ergebnis verändert das Modell.

Aber jede Erfahrung erweitert das Runtime-Wissen.

Zum Beispiel:

- erfolgreiche Strategien
- Fehlermuster
- neue Beziehungen
- bessere Skill-Kombinationen
- effizientere Abläufe

Nur freigegebene Informationen werden dauerhaft gespeichert.

Runtime-State Machine

Intern arbeitet der Algorithmus als Zustandsmaschine.

IDLE

↓

OBSERVE

↓

INTERPRET

↓

PLAN

↓

CHECK



EXECUTE



VERIFY



UPDATE MEMORY



IDLE

Dadurch besitzt jeder Agent jederzeit einen eindeutig definierten Zustand.

Parallelität

Mehrere Agenten können denselben Algorithmus gleichzeitig ausführen.

Jeder Agent besitzt jedoch:

- eigenen Kontext
- eigenes Gedächtnis
- eigene Ziele
- eigene Berechtigungen
- eigene Risiken

Die Runtime koordiniert lediglich die Zusammenarbeit.

Vorteile des OIDA-Algorithmus

Der Observe–Interpret–Decide–Act-Loop ermöglicht:

- ✓ nachvollziehbare Entscheidungen
 - ✓ reproduzierbare Abläufe
 - ✓ klare Zustände
 - ✓ kontrollierte Agenten
 - ✓ kontinuierliches Lernen
 - ✓ einfache Auditierbarkeit
 - ✓ modulare Erweiterbarkeit
 - ✓ stabile Multiagentenkoordination
-

Tech Club Erkenntnis

Die eigentliche Intelligenz eines Agenten entsteht nicht während der Ausführung.

Sie entsteht **zwischen Beobachtung und Handlung**.

Genau dort entscheidet sich,

welche Informationen relevant sind,

welche Risiken bestehen,

welche Ziele Priorität besitzen

und welche Handlung verantwortbar ist.

Der OIDA-Algorithmus bildet deshalb den universellen Herzschlag des Tech Club Agentic OS.

Jeder weitere Kern algorithmus – Gedächtnis, Skill-Auswahl, Risikoprüfung, Kommunikation oder Selbstanpassung – baut auf diesem kontinuierlichen Lebenszyklus auf.

Sonderheft II

Tech Club Agentic OS

Seite 5

Algorithmus II – Goal Priority Engine

Wie Agenten entscheiden, was zuerst getan werden muss

Ein Agent besitzt selten nur eine Aufgabe.

Während seiner Laufzeit können gleichzeitig eintreffen:

- neue Benutzeranfragen,
- API-Antworten,
- Erinnerungen,
- geplante Workflows,
- Warnmeldungen,
- Ereignisse anderer Agenten,
- Sicherheitsmeldungen,
- Systemänderungen.

Die eigentliche Herausforderung besteht daher nicht darin,
eine Aufgabe auszuführen.

Sondern die **richtige Aufgabe zum richtigen Zeitpunkt** auszuwählen.

Das Ziel des Goal Priority Engine

Der Goal Priority Engine entscheidet kontinuierlich:

Welche Aufgabe besitzt momentan die höchste Priorität?

Nicht jede Anfrage wird sofort bearbeitet.

Der Agent bewertet zunächst die Gesamtsituation.

Der Entscheidungsprozess

Jedes Ziel durchläuft dieselbe Bewertung.

Neue Aufgabe



Intent Analysis



Goal Extraction



Priority Calculation



Risk Evaluation



Resource Check



Dependency Analysis



Scheduling



Execution Queue

Erst danach gelangt die Aufgabe in die Runtime.

Mehrdimensionale Priorisierung

Tech Club verwendet keine einfache Prioritätszahl.

Jedes Ziel besitzt mehrere Bewertungsdimensionen.

Zum Beispiel:

Dringlichkeit

Wann muss die Aufgabe abgeschlossen sein?

Bedeutung

Welchen Nutzen besitzt das Ergebnis?

Risiko

Welche Folgen entstehen bei Fehlern?

Ressourcenbedarf

Welche Agenten,

Modelle,

Werkzeuge

oder Daten werden benötigt?

Abhängigkeiten

Welche anderen Aufgaben müssen zuerst abgeschlossen werden?

Vertrauensniveau

Sind genügend Informationen vorhanden,
um eine Entscheidung zu treffen?

Priorität ist dynamisch

Eine Aufgabe besitzt keine feste Priorität.

Sie verändert sich kontinuierlich.

Beispiel:

09:00

Task A

Priorität 5

↓

09:05

Neue Sicherheitswarnung

↓

Task B

Priorität 10

↓

Task B wird vorgezogen

Der Agent reagiert dadurch flexibel auf neue Ereignisse.

Konfliktlösung

Mehrere Ziele können gleichzeitig höchste Priorität besitzen.

Dann entscheidet die Runtime anhand zusätzlicher Regeln.

Zum Beispiel:

- Sicherheitsaufgaben vor Komfortfunktionen
- Menschliche Anfragen vor Hintergrundprozessen
- Compliance vor Automatisierung
- Kritische Systeme vor Analyseaufgaben

Diese Regeln können organisationsspezifisch angepasst werden.

Zusammenarbeit mit anderen Algorithmen

Der Goal Priority Engine arbeitet niemals allein.

Er kommuniziert ständig mit:

Memory Engine

↓

Risk Engine

↓

Skill Engine

↓

Compliance Kernel

↓

World State

↓

Task Scheduler

Dadurch entsteht eine konsistente Gesamtentscheidung.

Beispiel

Ein Benutzer fordert:

"Erstelle einen Bericht."

Währenddessen erkennt das System:

- neue DSGVO-Regel,
- Datenbankfehler,
- kritische API-Störung.

Der Goal Priority Engine entscheidet:

1. Sicherheitsprüfung
2. API-Stabilisierung
3. Datenvalidierung
4. Berichtserstellung

Der Benutzer erhält gleichzeitig eine transparente Information über die Verzögerung.

Vorteile

Der Algorithmus ermöglicht:

- ✓ dynamische Zielplanung
- ✓ konfliktfreie Entscheidungen
- ✓ effiziente Ressourcennutzung
- ✓ nachvollziehbare Priorisierung
- ✓ bessere Multiagentenkoordination

- ✓ geringere Wartezeiten
 - ✓ höhere Systemstabilität
 - ✓ reproduzierbare Entscheidungen
-

Vorbereitung auf Millionen Agenten

In einem zukünftigen Agentic OS arbeiten möglicherweise tausende Agenten gleichzeitig.

Der Goal Priority Engine verhindert,

dass alle Agenten dieselben Ressourcen gleichzeitig verwenden.

Er verteilt Arbeit intelligent,

koordiniert Abhängigkeiten

und verhindert unnötige Konflikte.

Dadurch bleibt das Gesamtsystem skalierbar.

Tech Club Erkenntnis

Intelligenz bedeutet nicht,

möglichst viele Aufgaben gleichzeitig zu bearbeiten.

Intelligenz bedeutet,

die wichtigste Aufgabe im richtigen Moment zu erkennen.

Der Goal Priority Engine bildet deshalb das strategische Planungszentrum des Tech Club Agentic OS.

Er verbindet Ziele,

Kontext,

Risiken,

Ressourcen,

Compliance

und Systemzustand zu einer gemeinsamen Prioritätsentscheidung.

Nicht Geschwindigkeit bestimmt die Qualität eines Agenten.

Sondern seine Fähigkeit, verantwortungsvoll zu entscheiden, welche Aufgabe jetzt wirklich wichtig ist.

Sonderheft II

Tech Club Agentic OS

Seite 6

Algorithmus III – Context-Aware Memory Retrieval

Intelligente Agenten benötigen kein großes Gedächtnis.

Sie benötigen das **richtige Gedächtnis zur richtigen Zeit**.

Die meisten heutigen KI-Systeme besitzen zwei Extreme.

Entweder:

Sie vergessen nahezu jede Unterhaltung nach kurzer Zeit.

Oder:

Sie laden sehr große Mengen an Informationen,

obwohl nur ein kleiner Teil tatsächlich relevant ist.

Beides ist ineffizient.

Tech Club Agentic OS verfolgt deshalb einen anderen Ansatz.

Nicht möglichst viel Speicher.

Sondern **kontextabhängigen Speicherzugriff**.

Erinnerung ist aktive Rekonstruktion

Auch das menschliche Gehirn arbeitet nicht wie eine Festplatte.

Menschen erinnern sich selten an vollständige Dokumente.

Sie erinnern sich an:

- Situationen,
- Beziehungen,
- Muster,
- Erfahrungen,
- Emotionen,
- Zusammenhänge.

Aus diesen Fragmenten entsteht die Erinnerung.

Tech Club überträgt dieses Prinzip auf Agenten.

Der Memory Retrieval Cycle

Jede Agentenanfrage beginnt mit einer Gedächtnisanalyse.

Current Situation

↓

Context Analysis

↓

Memory Query

↓

Graph Navigation

↓

Similarity Ranking

↓

Evidence Selection

↓

Context Assembly

↓

Decision Engine

Der Agent lädt nicht den gesamten Speicher.

Er lädt ausschließlich den aktuell relevanten Kontext.

Mehrschichtiger Speicher

Das Agentic OS unterscheidet verschiedene Gedächtnisebenen.

Runtime Memory

Aktuelle Sitzung.

Kurzfristige Informationen.

Working Memory

Aktuelle Aufgaben.

Zwischenergebnisse.

Temporäre Pläne.

Episodic Memory

Vergangene Projekte.

Frühere Entscheidungen.

Erfahrungen.

Semantic Memory

Fachwissen.

Regeln.

Dokumentationen.

Handbücher.

Procedural Memory

Skills.

Workflows.

Automatisierungen.

Algorithmen.

Organizational Memory

Unternehmensrichtlinien.

Compliance.

Dokumentationen.

Prozesswissen.

Erinnerung entsteht über Beziehungen

Der Agent sucht nicht nach Dateinamen.

Er sucht über den Wissensgraphen.

Beispiel:

Projekt Alpha

↓

API

↓

Python

↓

MCP Server

↓

Deployment

↓

Dokumentation

↓

Fehlerbericht

↓

Lösung

Dadurch werden Zusammenhänge sichtbar,

nicht nur Dateien.

Relevanzbewertung

Nicht jede Erinnerung besitzt denselben Wert.

Das System bewertet beispielsweise:

- Aktualität
- Vertrauensniveau
- Ähnlichkeit
- Projektrelevanz
- Benutzerkontext
- Quelle
- Version
- Häufigkeit der Nutzung

Erst danach wird entschieden,

welche Informationen tatsächlich geladen werden.

Adaptive Gedächtnis Größe

Der Agent bestimmt selbst,

wie viel Kontext erforderlich ist.

Eine einfache Frage benötigt vielleicht:

2 Dokumente.

Ein komplexes Projekt:

500 Wissensobjekte.

Dadurch bleibt das System effizient.

Zusammenarbeit mit dem Data Graph

Memory Retrieval arbeitet direkt mit:

Knowledge Graph

↓

Vector Search

↓

Semantic Index

↓

Document Store

↓

Equation Layer

↓

Trust Layer

Alle Speicherkomponenten liefern gemeinsam den benötigten Kontext.

Vorteile

Der Algorithmus ermöglicht:

- ✓ weniger Tonerverbrauch
- ✓ schnellere Antworten
- ✓ bessere Kontextqualität
- ✓ geringere Halluzinationen
- ✓ nachvollziehbare Quellen
- ✓ projektbezogenes Denken
- ✓ effiziente Agenten
- ✓ skalierbare Wissenssysteme

Grundlage für langfristiges Lernen

Jede neue Erfahrung wird bewertet.

Nicht jede Information wird dauerhaft gespeichert.

Das System entscheidet:

Ist diese Information:

- neu?
- relevant?
- vertrauenswürdig?
- wiederverwendbar?

Nur dann wird sie Teil des langfristigen Wissens.

Dadurch wird das Gedächtnis kontrolliert.

Nicht unendlich.

Tech Club Erkenntnis

Ein intelligenter Agent zeichnet sich nicht dadurch aus,

dass er alles weiß.

Sondern dadurch,

dass er **im richtigen Moment die richtige Erfahrung findet.**

Der Context-Aware Memory Retrieval Algorithmus verbindet deshalb Wissensgraph,

semantische Suche,

Vektorsuche,

Vertrauen Bewertung,

Versionsverwaltung

und Kontextanalyse

zu einem gemeinsamen Gedächtnissystem.

Nicht Datenmengen erzeugen Intelligenz.

Sondern die Fähigkeit, relevantes Wissen effizient zu finden, verantwortungsvoll zu nutzen und kontinuierlich weiterzuentwickeln.

Sonderheft II

Tech Club Agentic OS

Seite 7

Algorithmus IV – Skill Selection Engine

Fähigkeiten sind wichtiger als Programme

Klassische Software besitzt Funktionen.

Ein Agent besitzt Fähigkeiten.

Diese Fähigkeiten nennt Tech Club:

Skills

Ein Skill ist eine klar definierte,

versionierte,

überprüfbare

und wiederverwendbare Fähigkeit.

Zum Beispiel:

- PDF analysieren
- Python-Code ausführen
- GitHub lesen
- Datenbank abfragen
- E-Mail entwerfen
- Übersetzen
- Zusammenfassen
- Diagramm erzeugen
- API aufrufen
- Bild analysieren

Ein Agent besteht somit nicht aus einem großen Programm,

sondern aus vielen kleinen, kontrollierbaren Fähigkeiten.

Der Skill Selection Algorithmus

Vor jeder Handlung entscheidet der Agent,

welcher Skill tatsächlich benötigt wird.

Nicht jeder verfügbare Skill darf automatisch verwendet werden.

Goal



Intent Analysis



Capability Search



Skill Ranking



Permission Check



Risk Check



Version Check



Sandbox Preparation



Execute Skill



Result Validation

Erst nach erfolgreicher Prüfung beginnt die Ausführung.

Fähigkeiten statt monolithischer Systeme

Tech Club trennt bewusst:

Agent



Skill



Tool



Runtime



Ergebnis

Dadurch bleibt der Agent klein,

während seine Fähigkeiten kontinuierlich erweitert werden können.

Skill Registry

Alle verfügbaren Skills werden zentral verwaltet.

Jeder Skill besitzt beispielsweise:

Skill-ID

Name

Version

Beschreibung

Benötigte Rechte

Risikoklasse

Ein-/Ausgabe

Abhängigkeiten

Vertrauensstatus

Sandbox-Profil

Dadurch wird jede Fähigkeit nachvollziehbar.

Skill Discovery

Der Agent sucht nicht nach Dateinamen.

Er sucht nach Fähigkeiten.

Zum Beispiel:

"Ich benötige einen Skill,
der PDFs analysieren,
Tabellen erkennen
und Markdown erzeugen kann."

Die Runtime durchsucht automatisch die Skill Registry.

Mehrere Skills kombinieren

Komplexe Aufgaben bestehen meist aus mehreren Fähigkeiten.

Beispiel:

PDF lesen

↓

OCR

↓

Textanalyse

↓

Knowledge Graph

↓

Zusammenfassung

↓

Bericht erzeugen

Der Agent erzeugt daraus automatisch einen Workflow.

Secure Skill Sandbox

Jeder Skill wird isoliert ausgeführt.

Nicht direkt im Betriebssystem.

Sondern innerhalb einer kontrollierten Sandbox.

Die Sandbox begrenzt:

- Dateizugriffe
- Netzwerkzugriffe
- API-Zugriffe
- Laufzeit
- Speicherverbrauch
- Berechtigungen
- externe Prozesse

Dadurch kann ein fehlerhafter oder kompromittierter Skill das Gesamtsystem nicht gefährden.

Skill-Vertrauen

Nicht alle Skills besitzen dieselbe Vertrauensstufe.

Tech Club bewertet beispielsweise:

- Herkunft
- digitale Signatur
- Version
- Sicherheitsprüfung
- Teststatus
- Wartungsstatus
- bekannte Einschränkungen

Der Agent bevorzugt vertrauenswürdige Skills.

Zusammenarbeit mit Compliance

Vor jeder Ausführung kommuniziert die Skill Engine mit:

Compliance Kernel

↓

Risk Engine

↓

Permission Manager

↓

Model Registry

↓

Audit Logger

Nur wenn alle Prüfungen erfolgreich sind,
wird der Skill freigegeben.

Vorteile

Die Skill Selection Engine ermöglicht:

- ✓ modulare Erweiterbarkeit
 - ✓ sichere Ausführung
 - ✓ Wiederverwendbarkeit
 - ✓ versionierte Fähigkeiten
 - ✓ einfache Updates
 - ✓ nachvollziehbare Entscheidungen
 - ✓ geringere Angriffsfläche
 - ✓ modellunabhängige Agenten
-

Vorbereitung auf Agent Marketplaces

Langfristig können Unternehmen,

Universitäten

oder Open-Source-Communities

eigene Skills entwickeln.

Der Agent muss diese jedoch nicht kennen.

Er fragt lediglich:

Welche Fähigkeit wird benötigt?

Die Runtime findet automatisch den geeigneten Skill,

prüft Sicherheit,

Version,

Risiko

und Berechtigung

und integriert ihn in den aktuellen Workflow.

Tech Club Erkenntnis

Die Zukunft agentischer Systeme besteht nicht aus immer größeren Modellen.

Sie besteht aus **kleinen, spezialisierten, überprüfbaren Fähigkeiten**, die flexibel miteinander kombiniert werden können.

Der Skill Selection Algorithmus bildet deshalb das **Kompetenzzentrum des Tech Club Agentic OS**.

Er verbindet Ziele,

Kontext,

Vertrauen,

Compliance,

Sandboxing

und Fähigkeiten zu einer sicheren und skalierbaren Ausführungsarchitektur.

Nicht der Agent muss alles können.

Er muss wissen, welche Fähigkeit er wann, unter welchen Bedingungen und mit welchem Vertrauen einsetzen darf.

Sonderheft II

Tech Club Agentic OS

Seite 8

Algorithmus V – Runtime Risk Evaluation

Sichere Entscheidungen entstehen während der Laufzeit

Die meisten Softwaresysteme prüfen Berechtigungen nur beim Start einer Aktion.

Agentische Systeme benötigen deutlich mehr.

Während ihrer Laufzeit verändern sich kontinuierlich:

- Kontext,
- Daten,
- Benutzer,
- Berechtigungen,
- Ziele,
- Systemzustand,
- Risiken.

Eine ursprünglich sichere Aktion kann wenige Sekunden später ein anderes Risikoprofil besitzen.

Deshalb bewertet Tech Club Risiken **kontinuierlich während der gesamten Laufzeit**.

Der Runtime Risk Cycle

Jede geplante Aktion wird unmittelbar vor ihrer Ausführung bewertet.

Action Request

↓

Context Analysis



Risk Identification



Policy Evaluation



Impact Assessment



Human Oversight Check



Approve

oder

Modify

oder

Reject



Audit Log

Die Entscheidung erfolgt niemals ausschließlich aufgrund der ursprünglichen Anfrage.

Der aktuelle Systemzustand wird immer berücksichtigt.

Mehrdimensionale Risikobewertung

Die Runtime bewertet gleichzeitig mehrere Risikodimensionen.

Technisches Risiko

Kann diese Aktion das System beschädigen?

Datenschutzrisiko

Werden personenbezogene Daten verarbeitet?

Sind die erforderlichen Berechtigungen vorhanden?

Sicherheitsrisiko

Greift der Agent auf sensible Ressourcen zu?

Compliance-Risiko

Entspricht die Aktion den geltenden Richtlinien und regulatorischen Anforderungen?

Betriebsrisiko

Beeinflusst die Aktion andere Agenten oder kritische Prozesse?

Vertrauensrisiko

Sind die zugrunde liegenden Daten und Modelle ausreichend vertrauenswürdig?

Dynamische Risikoklassen

Die Bewertung ist nicht statisch.

Eine identische Aktion kann je nach Kontext unterschiedlich eingestuft werden.

Beispiel:

Ein Dokument zusammenzufassen,

das öffentlich verfügbar ist,

stellt meist ein geringes Risiko dar.

Die Zusammenfassung eines vertraulichen Personaldokuments kann dagegen zusätzliche Datenschutz- und Compliance-Prüfungen erfordern.

Nicht die Funktion allein,

sondern der **Anwendungskontext** bestimmt die Bewertung.

Zusammenarbeit mit dem Compliance Kernel

Die Risk Engine arbeitet eng mit anderen Komponenten zusammen.

Compliance Kernel

↓

Policy Engine

↓

Data Governance

↓

Model Registry

↓

Human Oversight

↓

Audit Logger

Erst das Zusammenspiel dieser Komponenten ermöglicht eine fundierte Entscheidung.

Unsicherheitsbewertung

Neben dem Risiko bewertet das System auch seine eigene Unsicherheit.

Zum Beispiel:

- unvollständiger Kontext
- widersprüchliche Informationen
- unbekannte Datenquellen
- geringe Modell Zuverlässigkeit
- fehlende Berechtigungen

Steigt die Unsicherheit über definierte Schwellenwerte,

kann das System:

- zusätzliche Informationen anfordern,
 - einen alternativen Workflow wählen,
 - oder eine menschliche Entscheidung einholen.
-

Adaptive Sicherheitsrichtlinien

Nicht jede Organisation besitzt dieselben Anforderungen.

Die Runtime lädt deshalb organisationsspezifische Richtlinien.

Zum Beispiel:

- Krankenhaus
- Universität
- Industrie
- Forschung
- öffentliche Verwaltung
- Finanzsektor

Jede Organisation kann eigene Risiko- und Freigaberegeln definieren, ohne den Agenten selbst verändern zu müssen.

Vorteile

Die Runtime Risk Evaluation ermöglicht:

- ✓ kontinuierliche Sicherheitsbewertung
 - ✓ kontextabhängige Entscheidungen
 - ✓ geringere Fehlentscheidungen
 - ✓ nachvollziehbare Freigaben
 - ✓ dynamische Policy-Anpassungen
 - ✓ regulatorische Unterstützung
 - ✓ sichere Multiagentensysteme
 - ✓ robuste Runtime-Architektur
-

Vorbereitung auf autonome Agentennetze

Je mehr Agenten parallel arbeiten,
desto wichtiger wird eine gemeinsame Risikobewertung.

Die Runtime Risk Evaluation verhindert,
dass lokale Entscheidungen unbeabsichtigt globale Auswirkungen erzeugen.

Sie sorgt dafür,
dass jede Handlung nicht nur technisch möglich,
sondern auch organisatorisch,
rechtlich
und sicherheitstechnisch verantwortbar ist.

Tech Club Erkenntnis

Autonomie bedeutet nicht,
dass ein Agent ohne Kontrolle handelt.

Autonomie bedeutet,
dass ein Agent **jede Handlung selbständig bewerten kann und gleichzeitig erkennt, wann menschliche Aufsicht erforderlich ist.**

Die Runtime Risk Evaluation bildet deshalb das **Sicherheitszentrum des Tech Club Agentic OS.**

Sie verbindet Kontext,

Compliance,

Datenschutz,

Vertrauen,

Unsicherheit

und organisatorische Richtlinien

zu einer kontinuierlichen Entscheidungsarchitektur.

Nicht jede mögliche Handlung wird ausgeführt.

Nur jene Handlung, die im aktuellen Kontext verantwortbar, nachvollziehbar und sicher ist.

Sonderheft II

Tech Club Agentic OS

Seite 9

Algorithmus VI – World-State Synchronization

Alle Agenten benötigen dieselbe Realität

Ein einzelner Agent kann mit seinem eigenen Wissensstand arbeiten.

Ein Multiagentensystem kann das nicht.

Sobald mehrere Agenten gleichzeitig arbeiten,

muss sichergestellt werden,

dass alle auf einem möglichst konsistenten Weltmodell arbeiten.

Nicht unbedingt auf identischen Daten,

aber auf einer gemeinsamen Realität.

Diese Aufgabe übernimmt der **World-State Synchronization Algorithm**.

Was ist der World State?

Der World State beschreibt den aktuellen Zustand des gesamten Systems.

Er umfasst beispielsweise:

- aktive Benutzer
- laufende Projekte

- Dokumentversionen
- verfügbare Skills
- Agentenzustände
- MCP-Server
- APIs
- Datenquellen
- Berechtigungen
- Policies
- Systemereignisse

Der World State ist keine einzelne Datenbank.

Er ist die aktuelle gemeinsame Sicht auf das Gesamtsystem.

Der Synchronisationszyklus

Jede relevante Änderung löst denselben Ablauf aus.

State Change

↓

Event Detection

↓

Affected Objects

↓

Graph Update

↓

Conflict Analysis

↓

Synchronization

↓

Agent Notification



Runtime Update



Audit

Dadurch reagieren alle Agenten auf dieselbe Veränderung.

Ereignisgesteuerte Architektur

Tech Club arbeitet ereignisorientiert.

Nicht jeder Agent fragt permanent alle Systeme ab.

Stattdessen entstehen Events.

Zum Beispiel:

Neue Datei



Skill aktualisiert



Policy geändert



Agent gestartet



API nicht erreichbar

↓

Benutzerrolle geändert

↓

Projekt abgeschlossen

Diese Ereignisse werden automatisch verteilt.

Der World Graph

Der World State basiert auf dem Wissensgraphen.

Nicht nur Objekte werden gespeichert.

Auch deren aktueller Zustand.

Zum Beispiel:

Project Alpha

Status:
Running

↓

Document

Version:
5

↓

Skill

Version:
2.4

↓

Agent

State:

Busy

↓

API

State:

Offline

Der Graph bildet dadurch die aktuelle Realität ab.

Konflikterkennung

Mehrere Agenten können gleichzeitig dieselbe Ressource verändern.

Zum Beispiel:

Agent A aktualisiert Dokument X.

Währenddessen erstellt Agent B ebenfalls Änderungen.

Der Synchronisationsalgorithmus erkennt:

- Versionskonflikte
- widersprüchliche Zustände
- doppelte Aufgaben
- konkurrierende Workflows

und entscheidet,

ob:

- zusammengeführt,
- verschoben,
- erneut ausgeführt

oder

- menschliche Hilfe angefordert wird.

Konsistenz statt Gleichzeitigkeit

Tech Club verfolgt nicht das Ziel,

jede Information sofort überall zu aktualisieren.

Entscheidend ist,

dass alle Agenten mit einer **konsistenten Sicht** arbeiten.

Je nach Anwendung können unterschiedliche Strategien genutzt werden:

- sofortige Synchronisation
- ereignisbasierte Aktualisierung
- periodische Synchronisation
- verteilte Konsensverfahren

Die Runtime wählt die geeignete Strategie.

Zusammenarbeit mit anderen Kernalgorithmen

Der World-State-Algorithmus kommuniziert permanent mit:

Memory Engine

↓

Goal Engine

↓

Skill Engine

↓

Risk Engine

↓

Compliance Kernel

↓

Communication Router

↓

Knowledge Graph

Dadurch besitzen alle Agenten denselben aktuellen Kontext.

Vorteile

Der Algorithmus ermöglicht:

- ✓ gemeinsame Realität
 - ✓ konsistente Agentenentscheidungen
 - ✓ weniger Konflikte
 - ✓ aktuelle Wissensgraphen
 - ✓ automatische Ereignisverarbeitung
 - ✓ skalierbare Multiagentensysteme
 - ✓ zuverlässige Workflowkoordination
 - ✓ bessere Fehlertoleranz
-

Grundlage für digitale Zwillinge

Langfristig beschreibt der World State nicht nur Software.

Er kann auch reale Systeme modellieren.

Zum Beispiel:

- Fabriken
- Smart Cities
- Forschungslabore
- Krankenhäuser
- Unternehmensprozesse
- Robotik
- IoT-Netzwerke

Die Agenten arbeiten dadurch nicht mehr nur mit Daten,
sondern mit einem ständig aktualisierten digitalen Abbild der Realität.

Tech Club Erkenntnis

Ein einzelner intelligenter Agent genügt nicht,
wenn alle Agenten unterschiedliche Wirklichkeiten sehen.

Die eigentliche Intelligenz eines Multiagentensystems entsteht erst,
wenn alle Beteiligten auf einer gemeinsamen,
vertrauenswürdigen

und kontinuierlich aktualisierten Weltbeschreibung arbeiten.

Der World-State Synchronization Algorithm bildet deshalb das **Koordinationszentrum des Tech Club Agentic OS**.

Er verbindet Wissensgraph,

Runtime,

Ereignisse,

Versionierung,

Kommunikation

und Synchronisation

zu einer gemeinsamen Realität für alle Agenten.

Nicht identische Daten sind entscheidend.

Entscheidend ist ein gemeinsames Verständnis des aktuellen Systemzustands.

Sonderheft II

Tech Club Agentic OS

Seite 10

Algorithmus VII – Self-Monitoring & Adaptive Runtime

Ein intelligenter Agent beobachtet nicht nur seine Umgebung.

Er beobachtet auch sich selbst.

Die meisten heutigen KI-Systeme bewerten lediglich ihre Ergebnisse.

Tech Club erweitert dieses Prinzip.

Jeder Agent überwacht während seiner gesamten Laufzeit:

- seine Entscheidungen,
- seine Ressourcen,
- seine Fehler,
- seine Unsicherheit,
- seine Leistung,
- seine Zielerreichung,
- und seine Einhaltung der Systemrichtlinien.

Dadurch entsteht ein selbstüberwachendes Runtime-System.

Der Self-Monitoring-Zyklus

Nach jeder abgeschlossenen Aktion beginnt automatisch eine interne Bewertung.

Action Finished



Collect Runtime Metrics



Evaluate Goal



Measure Confidence



Detect Errors



Detect Policy Violations



Adapt Strategy



Update Runtime Memory



Continue Runtime

Der Agent verbessert dadurch kontinuierlich sein Verhalten,
ohne seine grundlegenden Sicherheitsregeln selbstständig zu verändern.

Was wird überwacht?

Die Runtime sammelt permanent Kennzahlen.

Zum Beispiel:

Performance

- Laufzeit
 - Antwortzeit
 - CPU
 - Speicher
 - Netzwerk
-

Qualität

- Ziel erreicht?
 - Nutzer zufrieden?
 - Vollständigkeit
 - Genauigkeit
 - Konsistenz
-

Sicherheit

- Regelverletzungen
 - ungewöhnliche Aktionen
 - fehlgeschlagene Skills
 - kritische Warnungen
-

Vertrauen

- Quellenqualität
 - Modellvertrauen
 - Unsicherheitswert
 - Evidenzstärke
-

Adaptive Strategie

Der Agent darf seine Arbeitsstrategie anpassen.

Zum Beispiel:

Analyse zu langsam

↓

kleineren Skill wählen

API mehrfach nicht erreichbar

↓

Alternative Datenquelle wählen

Kontext unvollständig

↓

Benutzer nach weiteren Informationen fragen

Die Anpassung erfolgt innerhalb definierter Grenzen.

Nicht unbegrenzt.

Selbstdiagnose

Der Agent führt regelmäßig interne Diagnosen durch.

Beispielsweise:

- Sind alle Skills erreichbar?
- Sind Modelle verfügbar?
- Ist genügend Speicher vorhanden?
- Funktioniert die Kommunikation?
- Ist der World State aktuell?
- Sind Policies geladen?

- Sind Audit Logs vollständig?

Dadurch erkennt das System Probleme frühzeitig.

Zusammenarbeit mit Incident Response

Wird ein kritischer Fehler erkannt,

arbeitet der Agent automatisch mit dem Incident-System zusammen.

Anomalie erkannt



Runtime pausieren



Logs sichern



Compliance informieren



Administrator benachrichtigen



Recovery starten



Weiterbetrieb

Dadurch bleiben Fehler nachvollziehbar.

Grenzen der Selbstanpassung

Ein Agent darf **nicht**:

- seine Sicherheitsrichtlinien ändern,
- seine Risikoklasse verändern,
- Audit Logs löschen,
- Berechtigungen erweitern,
- Compliance-Regeln umgehen,
- Modelle eigenständig austauschen.

Diese Entscheidungen bleiben dem Systemadministrator oder autorisierten Personen vorbehalten.

Die Selbstanpassung betrifft ausschließlich die **Ausführungsstrategie**, nicht die Governance.

Zusammenarbeit mit allen Kernalgorithmen

Self-Monitoring verbindet:

Observe Loop

↓

Goal Engine

↓

Memory Engine

↓

Skill Engine

↓

Risk Engine

↓

World State

↓

Compliance Kernel

↓

Audit Logger

Dadurch entsteht ein kontinuierlicher Verbesserungsprozess.

Vorteile

Der Algorithmus ermöglicht:

- ✓ kontinuierliche Qualitätsverbesserung
 - ✓ frühzeitige Fehlererkennung
 - ✓ höhere Stabilität
 - ✓ bessere Ressourcennutzung
 - ✓ geringere Ausfallzeiten
 - ✓ nachvollziehbare Optimierung
 - ✓ adaptive Laufzeitstrategien
 - ✓ robuste Agentennetze
-

Gesamtbild des Agentic OS

Mit den ersten sieben Kernalgorithmen entsteht bereits eine vollständige Runtime-Architektur:

- **Observe–Interpret–Decide–Act** steuert den Lebenszyklus.
- **Goal Priority Engine** entscheidet über Prioritäten.
- **Context-Aware Memory Retrieval** liefert den passenden Kontext.
- **Skill Selection Engine** wählt sichere Fähigkeiten.
- **Runtime Risk Evaluation** bewertet Risiken kontinuierlich.
- **World-State Synchronization** sorgt für eine gemeinsame Realität.
- **Self-Monitoring & Adaptive Runtime** verbessert die Systemleistung innerhalb definierter Grenzen.

Diese Algorithmen arbeiten nicht unabhängig voneinander.

Sie bilden gemeinsam den **Agent Runtime Kernel** des Tech Club Agentic OS.

Tech Club Erkenntnis

Ein leistungsfähiger Agent zeichnet sich nicht dadurch aus,

dass er niemals Fehler macht.

Sondern dadurch,

dass er Fehler erkennt,

ihre Ursachen nachvollzieht,

seine Strategie verbessert,

und gleichzeitig die Grenzen seiner eigenen Verantwortung respektiert.

Das Tech Club Agentic OS entwickelt deshalb keine unkontrolliert selbstlernenden Agenten.

Es entwickelt **selbstüberwachende, nachvollziehbare und verantwortungsvoll adaptive Agenten**, deren Verbesserungen jederzeit auditierbar bleiben und deren Verhalten innerhalb klar definierter technischer und regulatorischer Grenzen erfolgt.

Nicht unkontrollierte Autonomie ist das Ziel.

Vertrauenswürdige, lernfähige und kontrollierbare Autonomie ist das Ziel.

